

 Politechnika Wrocławska	Projektowanie efektywnych algorytmów – projekt - zadanie 1	
	Maksymilian Tara 264000	Data oddania sprawozdania: 13.11.2023.
	Poniedziałek 15:15 – 16:55	Dr inż. Jarosław Mierzwa

1. Wstęp

Rozpatrywanym problemem, jest problem komiwojażera (ang. TSP – travelling salesman problem). Polega on, na znalezieniu minimalnego cyklu Hamiltona w pełnym grafie ważony. Cykl Hamiltona to taki cykl, w którym w grafie każdy wierzchołek odwiedzany jest dokładnie raz, a następnie następuje powrót do wierzchołka startowego. Minimalny – oznacza, że wagi przypisane do krawędzi grafu (połączeń między wierzchołkami) przez które przechodzi nasz znaleziony cykl, mają dawać najmniejszą możliwą sumę. Problem ten można przedstawić na przykładzie z życia jako podróżującego sprzedawcę, który musi odwiedzić każde z n miast na swojej określonej drodze (zaczyna z punktu A i na końcu chce znaleźć się w punkcie B) jak najniższym kosztem.

W tym eksperymencie problem rozpatrywany będzie dla grafów asymetrycznych – waga ścieżki (długość lub koszt drogi) z wierzchołka A do wierzchołka B, może być inna niż z wierzchołka B do wierzchołka A (w drugą stronę).

Problem TSP jest problemem NP-trudnym.

W celu rozwiązania zostały zaimplementowane 2 algorytmy: przegląd zupełny (brute force) oraz programowanie dynamiczne.

Złożoność obliczeniowa:

- Dla przeglądu zupełnego (brute force), dla grafu pełnego o n wierzchołkach, mamy $(n-1)!$ permutacji co daje nam złożoność czasową $O(n!)$. Złożoność pamięciowa prezentuje się

lepiej, ponieważ największa struktura danych wykorzystywana przez algorytm to macierz wag o rozmiarze n^2 .

- Dla programowania dynamicznego (dynamic programming) złożoność czasowa wynosi $O((n^2) * (2^n))$ co daje dużo mniejszy czas wykonywania algorytmu dla większych n (liczb wierzchołków). Niestety idzie z tym znaczne zwiększenie złożoności pamięciowej, która w tym przypadku wyniesie $O(n * (2^n))$.

2. Przykład Praktyczny

Przykład działania algorytmu programowania dynamicznego można zaprezentować na dwa sposoby – za pomocą rysunków grafów lub za pomocą wzoru. W tym przypadku użyjemy obydwóch dla lepszego zwizualizowania jego działania.

Wzór ten prezentuje się następująco:

$$g(i, S) = \min_{j \in S} \{L(i, j) + g(j, S - \{j\})\}$$

Gdzie:

$g(i, S)$ – Funkcja, oznacza najmniejszą sumę wag krawędzi, dla przejścia od wierzchołka i do wierzchołka startowego przy przejściu przez każdy wierzchołek ze zbioru S jeden raz.

i – Aktualnie odwiedzany wierzchołek

S – Zbiór wierzchołków pozostałych do odwiedzenia w aktualnej ścieżce

j – Wierzchołek, który został wybrany do odwiedzenia jako następny

$L(i, j)$ – Waga krawędzi z wierzchołka i do wierzchołka j , brana z macierzy wag

Jak możemy zauważyć, wzór ten jest rekurencyjny. Wnioskujemy więc, że ten algorytm obliczenia wag ścieżek wykonuje „od końca”.

Do przykładowego rozwiązania bierzemy graf o 4 wierzchołkach losowo wygenerowany przez program. Jako macierz wag prezentuje się on następująco:

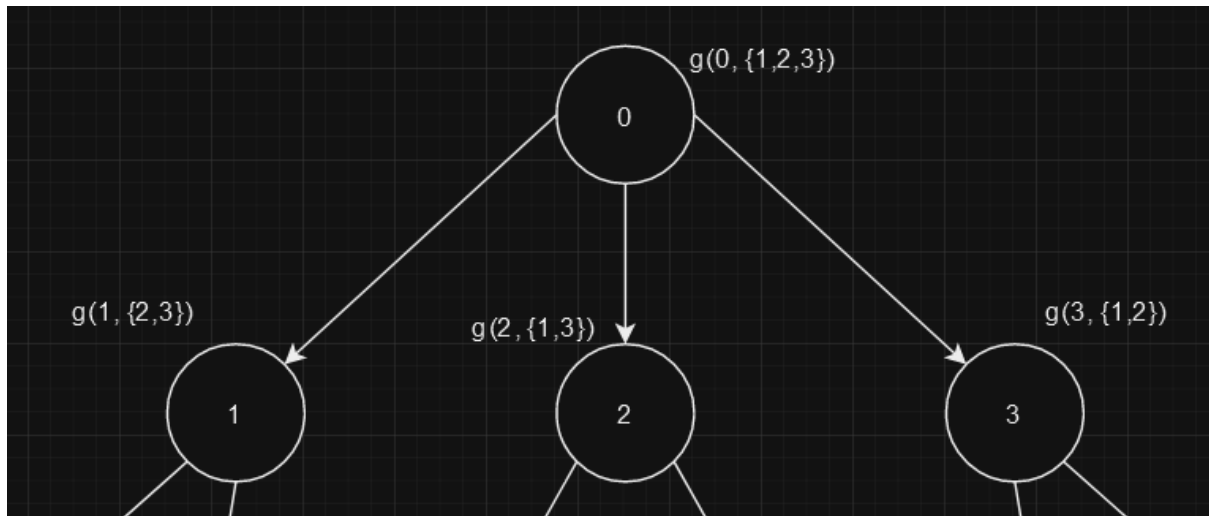
$$\begin{bmatrix} -1 & 9 & 14 & 2 \\ 21 & -1 & 12 & 27 \\ 15 & 16 & -1 & 15 \\ 18 & 19 & 10 & -1 \end{bmatrix}$$

-1 w macierzy oznaczają brak połączenia.

Za wierzchołek startowy przyjmijmy 0. Musimy zatem obliczyć $g(0, V - \{0\})$, gdzie V – zbiór wszystkich wierzchołków grafu.

Całość polega więc na znalezieniu $g(0, \{1,2,3\}) = \min_{j \in \{1,2,3\}} \{L(0, j) + g(j, \{1,2,3\} - \{j\})\}$. Ze względu na rekurencyjność wzoru, dzielimy go na mniejsze problemy, których rozwiązanie potrzebne jest do znalezienia pożądanego wyniku.

Rozwiązując dalej wzór rekurencyjny widzimy, że potrzebny nam jest najmniejszy wynik (min) z wszystkich możliwych permutacji połączeń wychodzących z wierzchołka startowego, które prezentują się następująco:



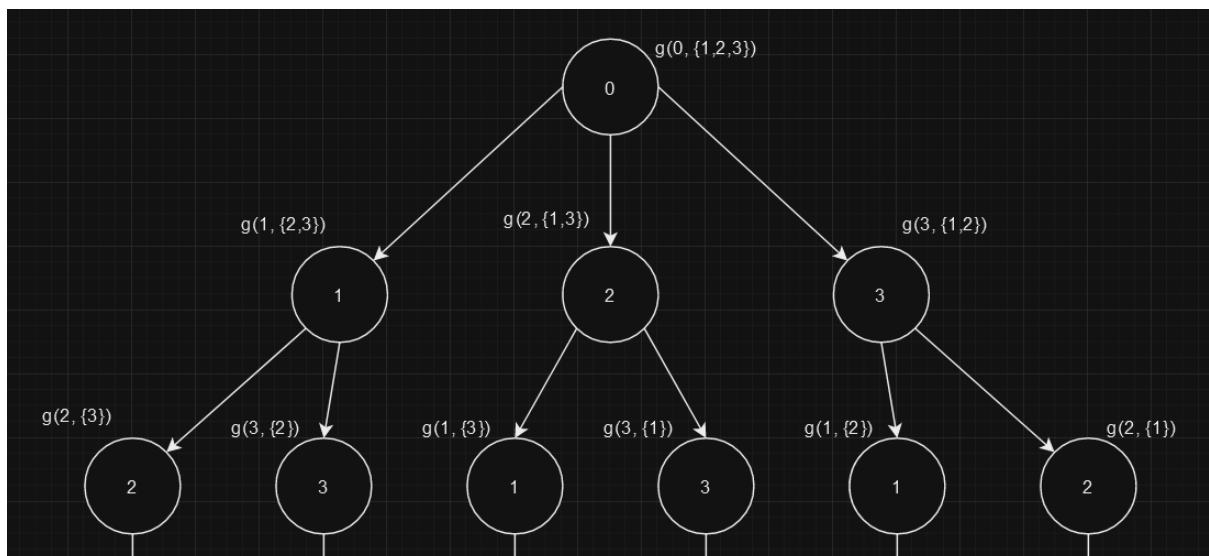
Jeśli rozpiszemy te wzory, otrzymamy dalsze kroki jakie musi wykonać nasz algorytm:

$$g(1, \{2, 3\}) = \min_{j \in \{2, 3\}} \{L(i, j) + g(j, \{2, 3\} - \{j\})\}$$

$$g(2, \{1, 3\}) = \min_{j \in \{1, 3\}} \{L(i, j) + g(j, \{1, 3\} - \{j\})\}$$

$$g(3, \{1, 2\}) = \min_{j \in \{1, 2\}} \{L(i, j) + g(j, \{1, 2\} - \{j\})\}$$

Zatem następnym etapem jest:



W listach wierzchołków do odwiedzenia zostało już tylko po jednym wierzchołku, biorąc to pod uwagę, po rozpisaniu wzorów daje nam to kolejno od lewej:

$$g(2, \{3\}) = L(i, j) + g(3, \emptyset)$$

$$g(3, \{2\}) = L(i, j) + g(2, \emptyset)$$

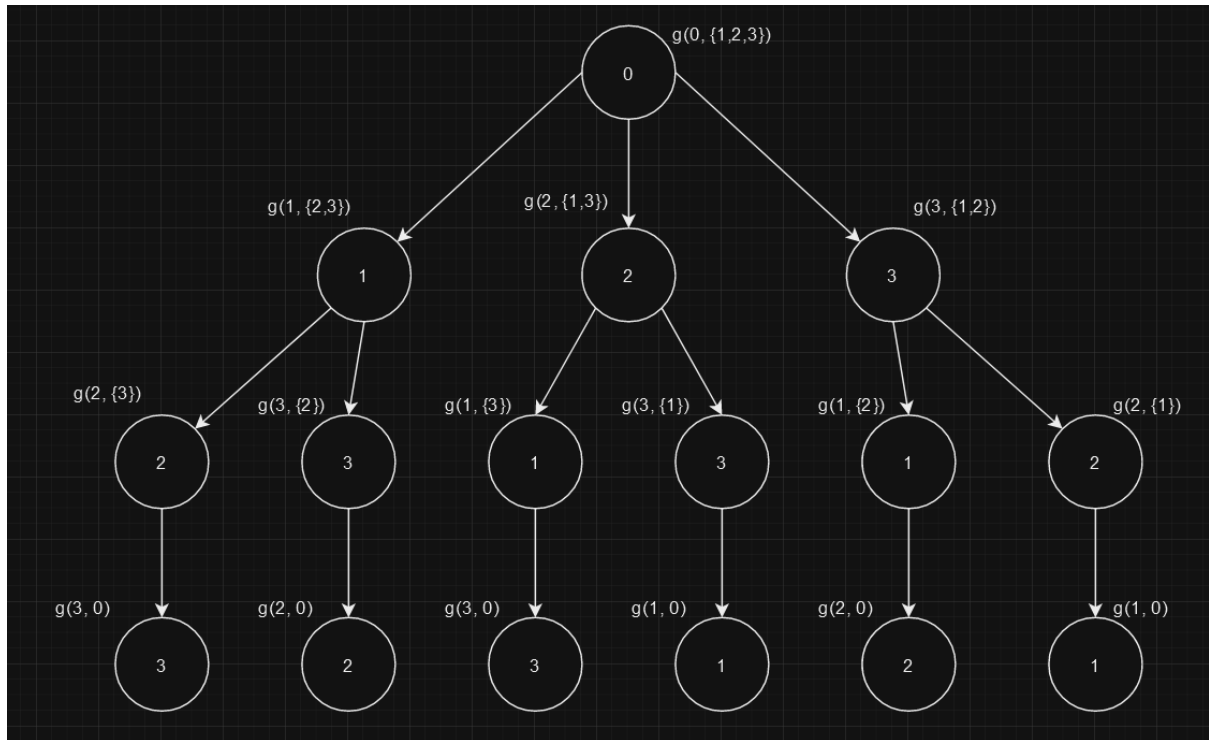
$$g(1, \{3\}) = L(i, j) + g(3, \emptyset)$$

$$g(3, \{1\}) = L(i, j) + g(1, \emptyset)$$

$$g(1, \{2\}) = L(i, j) + g(2, \emptyset)$$

$$g(2, \{1\}) = L(i, j) + g(1, \emptyset)$$

Następny etap wygląda zatem tak:



Ponieważ nie mamy już żadnych wierzchołków w zbiorze do odwiedzenia musimy zakończyć cykl powrotem do wierzchołka startowego, co oznacza też, że dotarliśmy do końca ścieżki – nie możemy już bardziej rozbić problemu na mniejsze i teraz jesteśmy w stanie odnaleźć rozwiązanie idąc „od końca”.

Zacznijmy więc obliczanie rozwiązań:

$$g(1, \emptyset) = L(1, 0) = 21$$

$$g(2, \emptyset) = L(1, 0) = 15$$

$$g(3, \emptyset) = L(1, 0) = 18$$

Wchodząc „wyżej” po grafie, podstawiamy znalezione wagi ścieżek do poprzednich wzorów:

$$g(2, \{3\}) = L(2, 3) + g(3, \emptyset) = 15 + 18 = 33$$

$$g(3, \{2\}) = L(3, 2) + g(2, \emptyset) = 19 + 15 = 34$$

$$g(1, \{3\}) = L(1, 3) + g(3, \emptyset) = 27 + 18 = 45$$

$$g(3, \{1\}) = L(3, 1) + g(1, \emptyset) = 19 + 21 = 40$$

$$g(1, \{2\}) = L(1, 2) + g(2, \emptyset) = 12 + 15 = 27$$

$$g(2, \{1\}) = L(2, 1) + g(1, \emptyset) = 16 + 21 = 37$$

Wracając jeszcze wyżej i ponownie podstawiając posiadane już wyniki, wybieramy mniejszą ze ścieżek i otrzymujemy:

$$g(1, \{2, 3\}) = \min_{j \in \{2, 3\}} \{L(i, j) + g(j, \{2, 3\} - \{j\})\} = \min\{L(1, 2) + g(2, \{3\}), L(1, 3) + g(3, \{2\})\} \\ = \min\{12 + 33, 27 + 34\} = 45 \text{ czyli ścieżka } 1, 2, 3, 0$$

$$g(2, \{1, 3\}) \\ = \min_{j \in \{1, 3\}} \{L(i, j) + g(j, \{1, 3\} - \{j\})\} = \min\{L(2, 1) + g(1, \{3\}), L(2, 3) + g(3, \{1\})\} = \min\{16 + 45, 15 + 40\} \\ = 55 \text{ czyli ścieżka } 2, 3, 1, 0$$

$$g(3, \{1, 2\}) = \min_{j \in \{1, 2\}} \{L(i, j) + g(j, \{1, 2\} - \{j\})\} = \min\{L(3, 1) + g(1, \{2\}), L(3, 2) + g(2, \{1\})\} \\ = \min\{19 + 27, 10 + 37\} = 46 \text{ czyli ścieżka } 3, 1, 2, 0$$

Jesteśmy zatem na ostatni kroku. Policzylismy już wszystkie pomniejsze problemy, które potrzebne były do znalezienia rozwiązania, i możemy podstawić je do głównego wzoru, z którego zaczynaliśmy:

$$g(0, \{1, 2, 3\}) = \min_{j \in \{1, 2, 3\}} \{L(i, j) + g(j, \{1, 2, 3\} - \{j\})\} \\ = \min\{L(0, 1) + g(1, \{2, 3\}), L(0, 2) + g(2, \{1, 3\}), L(0, 3) + g(3, \{1, 2\})\} \\ = \min\{9 + 45, 14 + 55, 2 + 46\} = 48 \text{ co daje nam ścieżkę } 0, 3, 1, 2, 0$$

Zakończyliśmy zatem algorytm z następującym wynikiem:

- Waga ścieżki = 48
- Ścieżka = 0, 3, 1, 2, 0

Co zgadza się również z wynikiem podanym przez program:

```
Sciezka:
[0] [3] [1] [2] [0]
Waga: 48
```

3. Opis Implementacji

Program został napisany w języku C++ i stosował się do podanych zaleceń:

- Wykorzystywane struktury danych są alokowane dynamicznie, w zależności do rozmiaru problemu,
- Program umożliwia weryfikację poprawności działania algorytmu, poprzez możliwość wczytania danych wejściowych (grafu i liczby wierzchołków) z pliku tekstowego. Pozwala to na porównanie wyniku i ścieżki podawanej przez program z grafu, dla którego rozwiązanie już znamy,
- Implementacje algorytmów zostały wykonane zgodnie z obiektowym paradygmatem programowania,
- Kod źródłowy był komentowany w celu zwiększenia przejrzystości i ułatwienia zrozumienia go,
- Menu programu umożliwia: Wczytanie danych z pliku, wygenerowanie losowych danych, wyświetlenie aktualnie załadowanego grafu, uruchomienie wybranego algorytmu (przeгляд zupełny albo programowanie dynamiczne).

- Program po wykonaniu algorytmu wyświetla następujące dane (wyniki): Długość ścieżki (suma wag krawędzi w tej ścieżce), ścieżka (ciąg odwiedzonych w niej wierzchołków) oraz czas wykonywania algorytmu,
- Do implementacji algorytmów nie zostały wykorzystane żadne biblioteki.

Dla algorytmu przeglądu zupełnego wykorzystywane są następujące struktury danych: dwuwymiarowa tablica liczb typu int o rozmiarze n^2 – macierz wag, tablica jednowymiarowa liczb typu int przechowująca aktualną ścieżkę o rozmiarze n , tablica jednowymiarowa liczb typu int przechowująca najkrótszą dotychczas ścieżkę o rozmiarze $n+1$ oraz tablica jednowymiarowa liczb typu int służąca jako „lista” odwiedzonych wierzchołków o rozmiarze n . Wykorzystane zostały również 3 liczby typu int.

Dla algorytmu programowania dynamicznego wykorzystywane są następujące struktury danych: dwuwymiarowa tablica liczb typu int o rozmiarze n^2 – macierz wag, dwie dwuwymiarowe tablice liczb typu int o rozmiarze $n \cdot (2^n)$ każda przechowujące: pierwsza - wagi policzonych permutacji ścieżek - i druga – rodziców każdego z wierzchołków w danej permutacji oraz tablica jednowymiarowa liczb typu int o rozmiarze $n+1$ przechowująca najkrótszą ścieżkę. Wykorzystana została również 1 liczba typu int do zapisania n (ilości wierzchołków/liczby miast).

Macierz, do której algorytm programowania dynamicznego w trakcie jego działania zapisuje policzone już wagi permutacji ścieżek jest na początku wypełniona liczbami „-1”. Na koniec iteracji algorytm zapisuje do tej macierzy wagę policzonej aktualnie ścieżki na odpowiadające jej miejsce. Na początku każdej iteracji algorytm sprawdza czy taki wariant ścieżki (wierzchołek, w którym aktualnie się znajdujemy i aktualny zbiór odwiedzonych wierzchołków) został już wcześniej policzony (czy na miejscu jemu odpowiadającym znajduje się inna liczba niż -1). Jeśli tak, algorytm nie musi już wykonywać wielu niepotrzebnych kroków tylko bierze znaną wcześniej wagę. Powoduje to znaczne zmniejszenie czasu wykonywania algorytmu w porównaniu do Przeglądu Zupełnego.

Odczytywanie ścieżki na koniec algorytmu umożliwia macierz w pewien sposób bliźniacza do poprzednio opisaney. Również jest ona na początek wypełniana liczbami „-1”, i algorytm na koniec swojego działania zapisuje do niej rodzica wierzchołka, do którego przechodzimy, w odpowiadającym mu miejscu (również zależy ono od wierzchołka, w którym aktualnie przebywamy i aktualnego stanu zbioru odwiedzonych wierzchołków). Jeśli nałożymy na siebie te 2 macierze, na tych samych miejscach, w których znajdują się wagi poszczególnych permutacji ścieżek, będą znajdować się numery wierzchołków, do których mamy przejść, jeśli chcemy podążać tą ścieżką.

Zbiór wierzchołków do odwiedzenia został zaimplementowany przy pomocy maski bitowej. Jest to po prostu liczba typu int, w której sprawdza się poszczególne bity w celu odczytania informacji o wierzchołkach, które już zostały odwiedzone, i które można odwiedzić. 1 – oznacza, że wierzchołek został już odwiedzony a 0 – że można go odwiedzić. Taka implementacja ma swoje plusy i minusy. Zdecydowaną przewagą nad implementacją przy pomocy tablicy lub listy jest dużo mniejsze zużycie pamięci. Minusem jednak jest możliwość wykonania algorytmu maksymalnie dla takiej liczby wierzchołków z jakiej ilości bitów składa się wykorzystany typ. Dla inta jest to 32.

4. Plan eksperymentu

Pomiary zostały wykonane dla $n \in \{7,8,9,10,11,12,13\}$ dla algorytmu przeglądu zupełnego oraz dla $n \in \{7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24\}$ dla algorytmu programowania dynamicznego. Czasem granicznym dla pomiarów było 60 sekund. Pomiary dla mniejszej ilości wartości n przy użyciu algorytmu przeglądu zupełnego wynikają ze zbyt dużego czasu wykonywania się tego algorytmu dla większych n – dla 14 wierzchołków algorytm wykonuje się około 220 sekund. Z kolei programowanie dynamiczne mierzone było dla wierzchołków aż do 24, ponieważ czas wykonania algorytmu dla 25 wierzchołków przekracza już wybrany czas graniczny.

Zbiór wartości n , dla których pomiary czasu wykonywania algorytmu programowania dynamicznego zawiera w sobie te wartości n , dla których były również wykonywane pomiary dla algorytmu brute force w celu dokładnego porównania działania tych dwóch algorytmów, jednak jest zwiększony w celu dalszego pokazania jego wydajności dla większej liczby wierzchołków.

Dla każdej wartości n generowane było 100 instancji problemu (100 różnych grafów). Wyniki brane pod uwagę w tym sprawozdaniu są wynikami uśrednionymi.

Mierzony był tylko czas wykonywania się algorytmu, bez czasu generowania grafu ani wypisywania wyników.

Sposób generowania danych do grafów:

```
29 void Matrix::GenerateRandomMatrix(int iloscWierzchołkow) {
30     iloscMiast = iloscWierzchołkow;
31     int value;
32
33     macierz = new int *[iloscMiast];
34     for (int i = 0; i < iloscMiast; i++) {
35         macierz[i] = new int[iloscMiast];
36     }
37
38     std::random_device rd;
39     std::mt19937 gen(sd: rd());
40     std::uniform_real_distribution<> distribution(a: 1, b: 30);
41
42     //wypełnienie macierzy liczbami wygenerowanymi losowo
43
44     for (int i = 0; i < iloscMiast; i++) {
45         for (int j = 0; j < iloscMiast; j++) {
46             if (i == j){
47                 macierz[i][j] = -1;
48                 continue;
49             }
50             value = distribution(& gen);
51             macierz[i][j] = value;
52         }
53     }
54 }
```

Wymagane było podanie liczby wierzchołków (n) dla generowanego grafu. Wykorzystana została do tego biblioteka <random>, która w bardzo prosty sposób umożliwia generowanie liczb z zadanego przedziału. Generowanie liczb były wpisywane po kolei na miejsca w macierzy wag, a jeśli aktualna „komórka” znajdowała się na jej przekątnej, wpisywana tam była -1, co symbolizowało brak połączenia z wierzchołkiem do tego samego wierzchołka (pętli w tym samym wierzchołku).

Metoda pomiaru czasu:

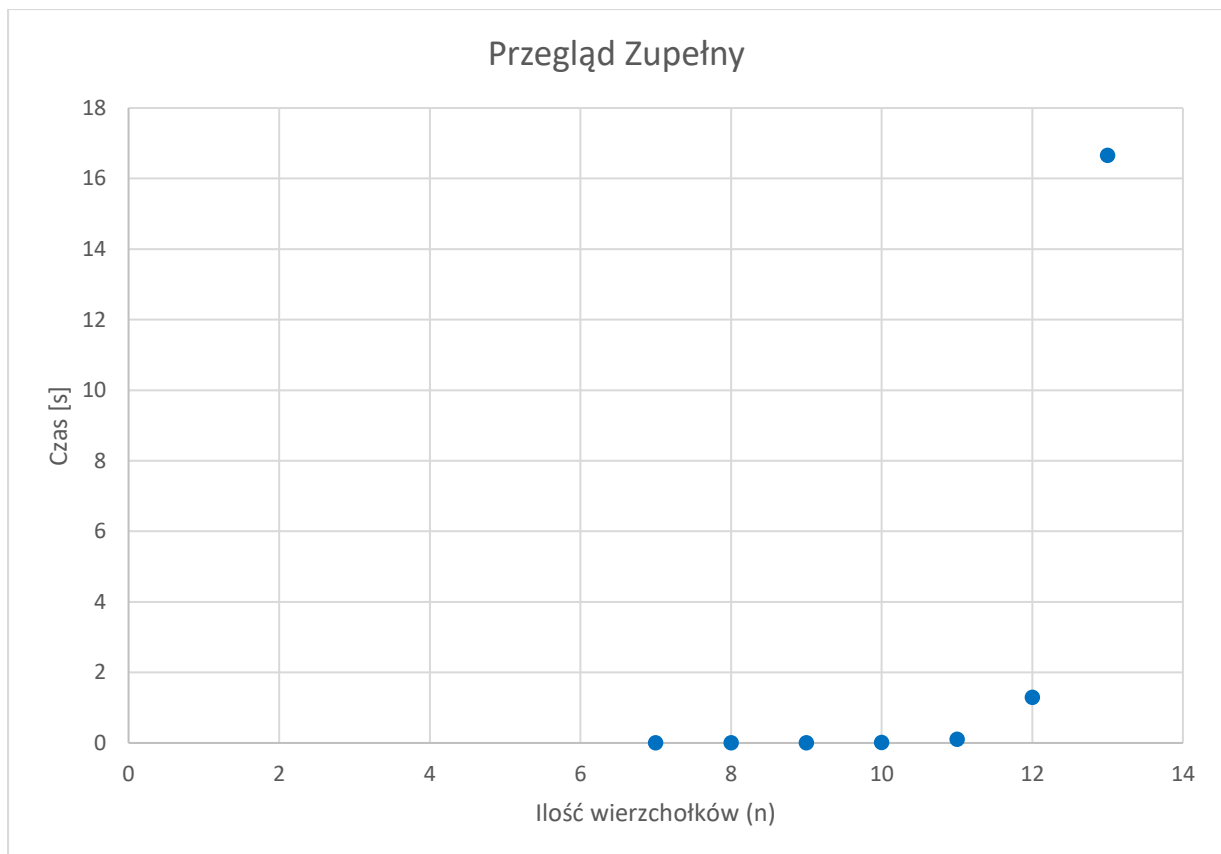
```
case 4: {
    BruteForce algorytmBT(macierzP);
    //pomiar
    auto start :time_point<...> = chrono::high_resolution_clock::now();
    //algorytm
    algorytmBT.runBruteForce(0);
    //pomiar
    auto stop :time_point<...> = chrono::high_resolution_clock::now();
    auto duration : duration<...> = chrono::duration_cast<chrono::microseconds>(d: stop - start);
    //rezultat
    algorytmBT.displayResults();
    cout << "Czas wykonania algorytmu: " << duration.count() << " us" << endl;
    break;
}
```

Wykorzystana została do tego biblioteka <chrono>, która w prosty sposób podkreślony na obrazku powyżej umożliwia pomiar czasu pomiędzy 2 zadanymi punktami w kodzie. Daje ona wynik o zadanej przez nas dokładności (możemy wybrać jednostkę czasu), w tym przypadku wybrane przeze mnie zostały mikrosekundy.

5. Wyniki eksperymentu

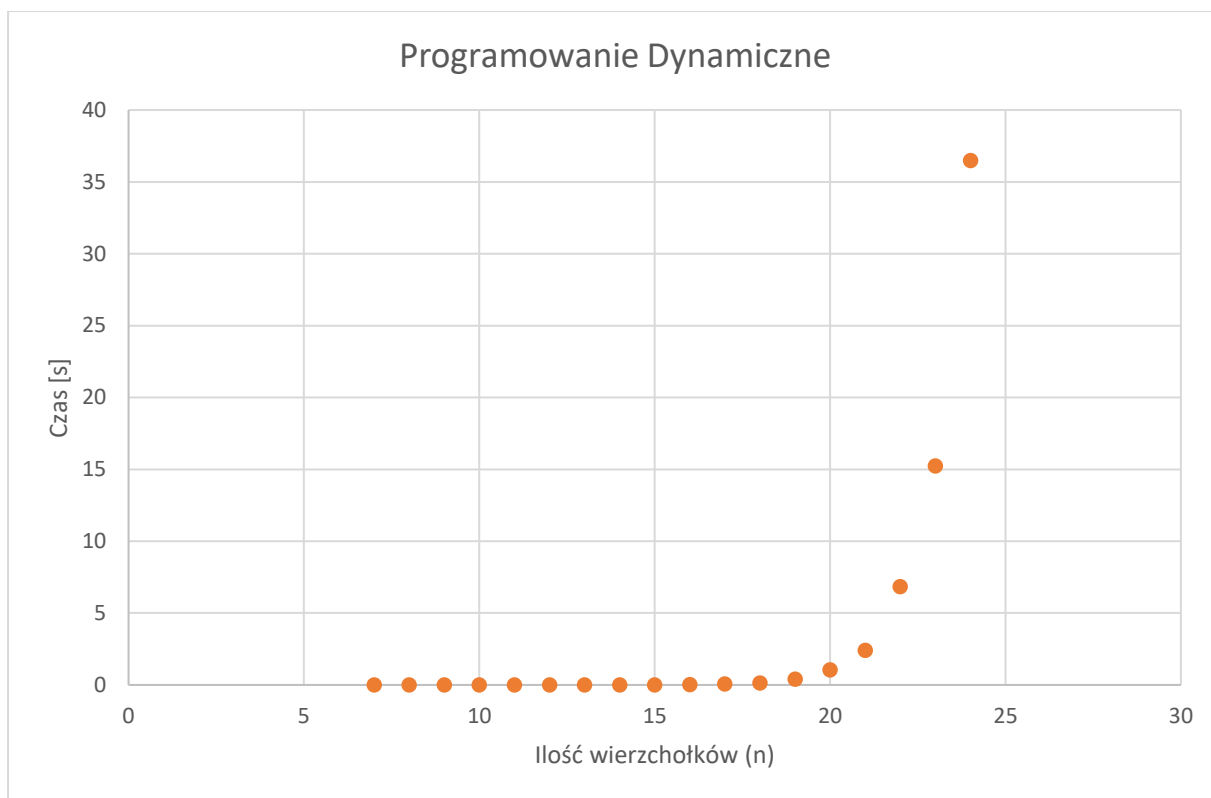
Przegląd Zupełny (Brute Force):

BT		
n	us	s
7	16	0.000016
8	116	0.000116
9	1071	0.001071
10	10141	0.010141
11	106063	0.106063
12	1296107	1.296107
13	16655319	16.65532

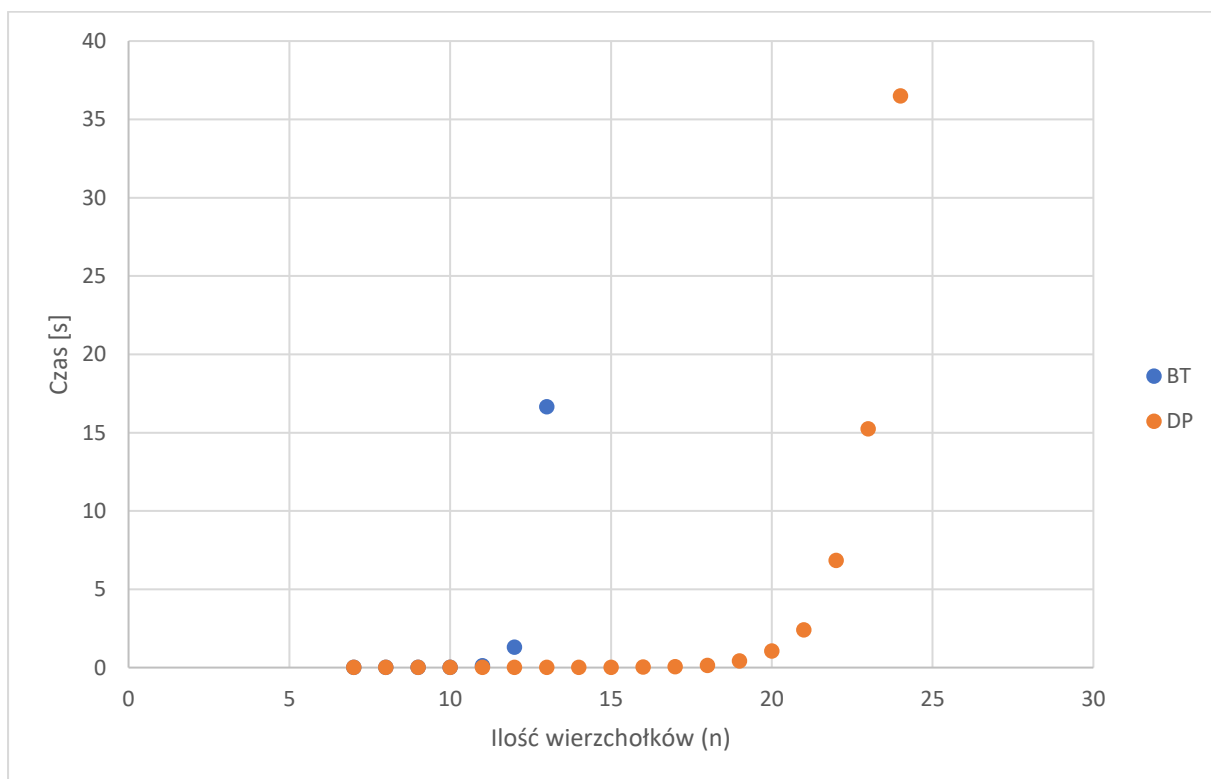


Programowanie Dynamiczne:

DP		
n	us	s
7	6	0.000006
8	17	0.000017
9	45	0.000045
10	116	0.000116
11	280	0.00028
12	668	0.000668
13	1684	0.001684
14	3702	0.003702
15	8389	0.008389
16	21648	0.021648
17	55521	0.055521
18	136209	0.136209
19	407561	0.407561
20	1040579	1.040579
21	2404208	2.404208
22	6829113	6.829113
23	15234544	15.23454
24	36483903	36.4839



Porównanie:



6. Wnioski

Jak możemy zauważyć, algorytm Programowania Dynamicznego działa dużo szybciej dla tych samych n w porównaniu do algorytmu Przeglądu zupełnego. Co więcej, jego czas wykonywania nie rośnie tak szybko wraz z liczbą wierzchołków, jak dla algorytmu przeglądu zupełnego.

Zdecydowanie bardziej wartym uwagi algorytmem wydaje się Programowanie Dynamiczne. Ma ono jednak jedną zauważalną wadę. Jest nią złożoność pamięciowa. Przy wykonywaniu testów i determinowaniu, dla jakich wartości n czasy wykonywania mieszczą się w ustalonym czasie granicznym, dla $n = 26$ zajęty został cały dostępny ram na moim urządzeniu – 15.9/16GB. Próby wykonania algorytmu dla wyższych n kończyły się niepowodzeniem. Widzimy więc na realnym przypadku, że dla bardziej rozległych grafów potrzebne by były zaawansowane urządzenia. Jednak w porównaniu z Brute Forcem, czas wykonania Programowania Dynamicznego dla większych ilości wierzchołków jest zdecydowanie krótszy. Dla $n=26$ czas wykonania algorytmu Brute Force byłby tak duży, że nie byłoby możliwe zakończenie go w realnym czasie nawet na bardzo wydajnych jednostkach. Wynika to po prostu ze zbyt dużej liczby permutacji. Przy tym ograniczenia sprzętowe wynikające ze zbyt małej ilości pamięci RAM są prostsze do zlikwidowania.

Bibliografia

<https://www.math.nagoya-u.ac.jp/~richard/teaching/s2020/Quang1.pdf>

<https://www.youtube.com/watch?v=Q4zHb-Swzro&t=363s>

https://drive.google.com/file/d/1Ug1RhLB6lhsXUCIv2eUyf_pYsdelZc-J/view?usp=drive_link

https://en.wikipedia.org/wiki/Travelling_salesman_problem